

**INSTITUTO
FEDERAL**
Santa Catarina

Objetos JavaScript

Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO
COMPUTACIONAL E
ALGORITMOS

Análise e Desenvolvimento
de Sistemas

Referência

definição

- **Em JavaScript**, ao manipular tipos de dados complexos, como **objetos** (`{}`) e **arrays** (`[]`), o valor não é copiado diretamente.
- Nesses casos, é criada uma **referência** para o local da memória onde o dado original está armazenado.

exemplos

```
let lista = [1, 2, 3];  
let outraLista = lista;
```

```
lista.push(4);
```

```
console.log(outraLista); // [1, 2, 3, 4]
```



Referência

definição

- **Em JavaScript**, ao manipular tipos de dados complexos, como **objetos** (`{}`) e **arrays** (`[]`), o valor não é copiado diretamente.
- Nesses casos, é criada uma **referência** para o local da memória onde o dado original está armazenado.

exemplos

```
let a = 10;  
let b = a;  
b = 20;
```

```
console.log(a); // 10  
console.log(b); // 20
```



Template Literals (Strings)

definição

- **interpolação de strings com expressões JS**
- pode incluir qualquer número de expressões
- pode abranger qualquer número de linhas
- delimitada pelo símbolo da crase (`)
- expressões delimitadas por `${ }`

exemplos

```
let nome = "Alice";
```

```
let saudacao = `Olá, ${nome}!`;
```

```
console.log(saudação); // Olá, Alice!
```



Objetos

definição

- **agregados de múltiplos valores**

exemplos



Objetos

definição

- agregados de múltiplos valores
- **coleções não ordenadas de propriedades, cada uma com um nome e um valor**

exemplos



Objetos

definição

- agregados de múltiplos valores
- coleções não ordenadas de propriedades, cada uma com um nome e um valor

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

JSON!

Objetos

definição

- agregados de múltiplos valores
- coleções não ordenadas de propriedades, cada uma com um nome e um valor
- valores podem ser primitivos ou objetos

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```



Objetos

definição

- agregados de múltiplos valores
- coleções não ordenadas de propriedades, cada uma com um nome e um valor
- valores podem ser primitivos ou objetos
- manipulados por referência

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```



Objetos

definição

- agregados de múltiplos valores
- coleções não ordenadas de propriedades, cada uma com um nome e um valor
- valores podem ser primitivos ou objetos
- manipulados por referência

exemplos

```
let x = {  
  nome: "João",  
  idade: 25  
}  
  
console.log(x);  
// { nome: 'João', idade: 25 }  
  
let y = x;  
  
y.nome = "Maria";  
  
console.log(x);  
// { nome: 'Maria', idade: 25 }
```



Objetos

propriedades

- **nomes das propriedades são strings**

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

Objetos

propriedades

- nomes das propriedades geralmente são strings
- **qualquer string pode ser nome de propriedades, mesmos strings vazias**

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

Objetos

propriedades

- nomes das propriedades geralmente são strings
- qualquer string pode ser nome de propriedades, mesmos strings vazias
- **objetos não podem ter duas propriedades com o mesmo nome**

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

Objetos

propriedades

- nomes das propriedades geralmente são strings
- qualquer string pode ser nome de propriedades, mesmos strings vazias
- objetos não podem ter duas propriedades com o mesmo nome
- **propriedades podem ser adicionadas e excluídas**

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

Objetos

propriedades

- nomes das propriedades geralmente são strings
- qualquer string pode ser nome de propriedades, mesmos strings vazias
- objetos não podem ter duas propriedades com o mesmo nome
- propriedades podem ser adicionadas e excluídas
- **valores podem ser primitivos ou objetos**

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```



Objetos

criando objetos

- como literais

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```



Objetos

criando objetos

- como literais

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

```
let vazio = {};
```

```
let ponto = { x: 0, y: 0 };
```

```
let outroPonto = { x: ponto.x, y: ponto.y+1 };
```



Objetos

criando objetos

- como literais
 - lista de pares nome-valor separados por vírgula
 - nomes e valores separados por dois pontos (:)
 - os valores podem ser qualquer expressão JS
 - tudo deve estar entre chaves ({})

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

```
let vazio = {};
```

```
let ponto = { x: 0, y: 0 };
```

```
let outroPonto = { x: ponto.x, y: ponto.y+1 };
```



Objetos

criando objetos

- como literais
- com o operador new

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

```
let vazio = {};
```

```
let ponto = { x: 0, y: 0 };
```

```
let outroPonto = { x: ponto.x, y: ponto.y+1 };
```



Objetos

criando objetos

- como literais
- com o operador new

exemplos

```
let o = new Object();  
// Cria um objeto vazio: o mesmo que {}.
```

```
let a = new Array();  
// Cria um array vazio: o mesmo que [].
```

```
let d = new Date();  
// Cria um objeto Date representando o horário  
atual
```

```
let r = new Map();  
// Cria um objeto Map para mapeamento de  
chave/valor
```



Objetos

criando objetos

- como literais
- com o operador new
- com **Object.create()**

exemplos

```
let o = new Object();  
// Cria um objeto vazio: o mesmo que {}.
```

```
let a = new Array();  
// Cria um array vazio: o mesmo que [].
```

```
let d = new Date();  
// Cria um objeto Date representando o horário  
atual
```

```
let r = new Map();  
// Cria um objeto Map para mapeamento de  
chave/valor
```



Objetos

criando objetos

- como literais
- com o operador new
- com `Object.create()`
 - cria objeto a partir de um protótipo

exemplos

```
let o = new Object();  
// Cria um objeto vazio: o mesmo que {}.
```

```
let a = new Array();  
// Cria um array vazio: o mesmo que [].
```

```
let d = new Date();  
// Cria um objeto Date representando o horário  
atual
```

```
let r = new Map();  
// Cria um objeto Map para mapeamento de  
chave/valor
```



Objetos

criando objetos

- como literais
- com o operador new
- com `Object.create()`
 - cria objeto a partir de um protótipo

exemplos

```
let objeto = Object.create({x: 1, y: 2});
```

```
let vazio = Object.create(Object.prototype);  
// vazio é como {} ou new Object().
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```



Objetos

acessando propriedades

- **operadores ponto (.) e colchetes ([])**
 - ao usar colchetes, as expressões devem resultar em strings

exemplos

```
let livro = {  
  titulo: "JavaScript",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "David",  
    sobrenome: "Flanagan"  
  },  
  ano: 2011  
};
```

Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
 - ao usar colchetes, as expressões dentro deles devem sempre resultar em uma string

exemplos

```
let livro = {
  titulo: "JavaScript",
  subtítulo: "O Guia Definitivo",
  para: "todas as audiências",
  autor: {
    "primeiro nome": "David",
    sobrenome: "Flanagan"
  },
  ano: 2011
};
```

```
let autor = livro.autor;
autor.sobrenome = "de Assis";
autor["primeiro nome"] = "Machado";
livro["titulo"] = "Dom Casmurro";
console.log(livro); // ???
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
 - ao usar colchetes, as expressões dentro deles devem sempre resultar em uma string

exemplos

```
let autor = livro.autor;  
autor.sobrenome = "de Assis";  
autor["primeiro nome"] = "Machado";  
livro["titulo"] = "Dom Casmurro";  
console.log(livro);  
  
{  
  titulo: "Dom Casmurro",  
  subtitulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011  
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- **propriedades inexistentes são criadas automaticamente quando escritas**

exemplos

```
let autor = livro.autor;  
autor.sobrenome = "de Assis";  
autor["primeiro nome"] = "Machado";  
livro["titulo"] = "Dom Casmurro";  
console.log(livro);
```

```
{  
  titulo: "Dom Casmurro",  
  subtitulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011  
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas

exemplos

```
livro.edicao = 7;  
console.log(livro);  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
  edicao: 7  
}
```

Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- **pode ser excluídas com o comando delete**

exemplos

```
livro.edicao = 7;  
console.log(livro);  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
  edicao: 7  
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete

exemplos

```
delete livro.ano;  
console.log(livro);  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  edicao: 7  
}
```

Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete
- quando propriedades não existem

exemplos

```
delete livro.ano;  
console.log(livro);  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  edicao: 7  
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete
- quando propriedades não existem

exemplos

```
let genero = livro.autor.genero;  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete
- quando propriedades não existem

exemplos

```
let genero = livro.autor.genero; // undefined

{
  titulo: "Dom Casmurro",
  subtítulo: "O Guia Definitivo",
  para: "todas as audiências",
  autor: {
    "primeiro nome": "Machado",
    sobrenome: "de Assis"
  },
  ano: 2011,
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete
- quando propriedades não existem

exemplos

```
let genero = livro.autor.genero.length;  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
}
```

Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete
- quando propriedades não existem

exemplos

```
let genero = livro.autor.genero.length;  
// TypeError: Cannot read properties of  
undefined (reading 'length')  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete
- quando propriedades não existem
- **podem ser acessadas condicionalmente (?. e ?.[])**

exemplos

```
let genero = livro.autor.genero.length;  
// TypeError: Cannot read properties of  
undefined (reading 'length')  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
}
```



Objetos

acessando propriedades

- operadores ponto (.) e colchetes ([])
- propriedades inexistentes são criadas automaticamente quando escritas
- pode ser excluídas com o comando delete
- quando propriedades não existem
- podem ser acessadas condicionalmente (?. e ?.[])

exemplos

```
let genero = livro.autor?.genero;  
let nome = livro.autor?.["nome"];  
  
{  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
}
```



Objetos

iterando sobre as propriedades dos objetos

- **for-in** permite a repetição da execução de um bloco de instruções para cada propriedade de um objeto

exemplos

```
let livro = {  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
  edicao: 7  
};
```



Objetos

iterando sobre as propriedades dos objetos

- **for-in** permite a repetição da execução de um bloco de instruções para cada propriedade de um objeto
 - **observação** enquanto for-of permite percorrer apenas objetos iteráveis, for-in pode percorrer qualquer objeto

exemplos

```
let livro = {  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
  edicao: 7  
};
```

Objetos

iterando sobre as propriedades dos objetos

- **for-in** permite a repetição da execução de um bloco de instruções para cada propriedade de um objeto
 - **observação** enquanto for-of permite percorrer apenas objetos iteráveis, for-in pode percorrer qualquer objeto

exemplos

```
let livro = {  
  titulo: "Dom Casmurro",  
  subtítulo: "O Guia Definitivo",  
  para: "todas as audiências",  
  autor: {  
    "primeiro nome": "Machado",  
    sobrenome: "de Assis"  
  },  
  ano: 2011,  
  edicao: 7  
};  
  
for (let chave in livro) {  
  console.log(chave + ": " + livro[chave]);  
}
```



Objetos

métodos

- **são ações que podem ser realizadas em objetos**



Objetos

métodos

- são ações que podem ser realizadas em objetos
- **em JS, são funções armazenadas como valores de propriedades**



Objetos

métodos

- são ações que podem ser realizadas em objetos
- **em JS, são funções armazenadas como valores de propriedades**
 - declaração padrão

exemplos

```
let retangulo = {  
  largura: 7,  
  altura: 9,  
  getArea : function() {  
    return this.largura * this.altura;  
  }  
};
```



referência ao próprio objeto!

Objetos

métodos

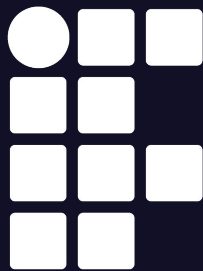
- são ações que podem ser realizadas em objetos
- **em JS, são funções armazenadas como valores de propriedades**
 - declaração padrão
 - declaração moderna

exemplos

```
let retangulo = {  
  largura: 7,  
  altura: 9,  
  getArea() {  
    return this.largura * this.altura;  
  }  
};
```

Exercícios

Resolva a lista de exercícios do SIGAA



**INSTITUTO
FEDERAL**
Santa Catarina

Objetos JavaString

Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO
COMPUTACIONAL E
ALGORITMOS

Análise e Desenvolvimento
de Sistemas